Project Design Phase-II

Technology Stack (Architecture & Stack)

Date	31 January 3035
Team ID	GACSE-077
Project Name	FitBuddy – AI-Powered Fitness Planner
Maximum Marks	4 Marks

Technical Architecture

Architecture Overview

FitBuddy is a full-stack web application designed to generate personalized fitness workout plans using artificial intelligence. The system integrates a web interface, backend application logic, AI model integration, and database storage to deliver an interactive and adaptive fitness planning experience.

The application follows a layered architecture in which different responsibilities are separated into logical layers. This approach improves maintainability, modularity, and scalability of the system.

The major architectural layers of the system include:

1. Presentation Layer – Handles user interaction through web pages where users provide their fitness details and view generated workout plans.
2. Application Logic Layer – Processes user inputs, manages application routes, and coordinates the generation and updating of workout plans.
3. AI Integration Layer – Communicates with the Google Gemini API to generate personalized workout routines and refine them based on user feedback.
4. Data Management Layer – Stores user data and generated plans using a lightweight relational database.

5. External API Layer – Connects the application with the Gemini large language model for AI-based workout generation.
6. Deployment Infrastructure – Supports running the application on a local server during development or cloud hosting environments for deployment.

Architectural Characteristics

The FitBuddy system is designed with the following characteristics:

- The frontend interface is built using HTML templates rendered through Jinja2.
- Application logic and routing are implemented using the FastAPI framework in Python.
- Personalized workout plans are generated using the Google Gemini AI model through the google-genai API.
- User fitness data and workout plans are stored in a SQLite database managed with SQLAlchemy ORM.
- The application follows a modular backend architecture with separate modules for AI processing, database management, and routing logic.
- Environment variables are used to securely store sensitive information such as API keys.

Logical Architecture Breakdown

Local Components

- FastAPI Web Application
- Backend Processing Modules
- SQLite Database
- Workout Plan Formatting Engine
- Nutrition Tip Generator

External Interfaces

- Google Gemini API (AI workout plan generation and refinement)

Infrastructure Demarcation

Layer	Deployment Location
User Interface	Web Browser
Application Logic	Local Server / Cloud Server
AI Model	Cloud (Google Gemini API)
Database Storage	Local SQLite Database
External API Integration	Cloud-based AI Service

Table 1: Components & Technologies

S.No	Component	Description	Technology
1	User Interface	Web-based interface for user input and workout plan display	HTML, CSS, JavaScript, Jinja2
2	Application Logic – 1	API routing and request handling	FastAPI (Python)
3	Application Logic – 2	Workout plan generation and feedback processing	Python
4	Application Logic – 3	AI integration and prompt handling	Python + Google Gemini API
5	Database	Persistent storage of user profiles and generated workout plans	SQLite + SQLAlchemy
6	Cloud Database	Not implemented in the current version	—

S.No	Component	Description	Technology
7	File Storage	Static assets such as CSS and JavaScript files	Local File System
8	External API – 1	AI workout plan generation and plan updates	Google Gemini API
9	External API – 2	Not applicable in the current version	—
10	Machine Learning Model	Generates personalized workout plans based on user input	Google Gemini Large Language Model
11	Infrastructure (Server / Cloud)	Deployment of FastAPI application	Local System / Cloud Hosting

---

Table 2: Application Characteristics

S.No	Characteristics	Description	Technology
1	Open-Source Frameworks	Frameworks used for application development	FastAPI, SQLAlchemy, Jinja2
2	Security Implementations	Secure handling of API keys and configuration variables	Environment Variables, python-dotenv
3	Scalable Architecture	Modular backend design allows integration of new AI models and databases in the future	Python Modular Architecture
4	Availability	Web-based application accessible through browser interface	Local Server / Cloud Deployment
5	Performance	Lightweight backend with efficient API communication and local database queries	Python Optimized Processing